



Multitask reinforcement learning with metadata-guided adaptive routing

Rui Pan ^{a,1}, Haoran Luo ^{b,1}, Quan Yuan ^{a,*}, Guiyang Luo ^a, Jinglin Li ^a,
Tiesunlong Shen ^{c,*}, Rui Mao ^b, Erik Cambria ^b

^a School of Computer Science, Beijing University of Posts and Telecommunications, Beijing, 100876, China

^b College of Computing and Data Science, Nanyang Technological University, 639798, Singapore

^c Duke-NUS Medical School, 169857, Singapore

ARTICLE INFO

Keywords:

Multitask reinforcement learning
negative knowledge transfer
metadata
adaptive routing

ABSTRACT

Multitask reinforcement learning aims to train a unified policy that generalizes across multiple related tasks, improving sample efficiency and promoting knowledge transfer. However, existing methods often suffer from negative knowledge transfer due to task interference, especially when using hard parameter sharing across tasks with diverse dynamics or goals. Conventional solutions typically adopt shared backbones with task-specific heads, gradient projection methods, or routing-based networks to mitigate conflict. However, many of these methods rely on simplistic task identifiers (e.g., one-hot vectors), lack expressive representations of task semantics, or fail to modulate shared components in a fine-grained, task-specific manner. To overcome these challenges, we propose **Metadata-guided Adaptive Routing (MetaAR)**, a novel framework that incorporates rich task metadata such as natural language descriptions to generate expressive and interpretable task representations. These representations are injected into a dynamic routing network, which adaptively reconfigures layer-wise computation paths in a shared modular policy network. To enable robust task-specific adaptation, we further introduce a noise-injected Top-K routing mechanism that dynamically selects the most relevant computation paths for each task. By injecting stochasticity during routing, this mechanism promotes exploration and mitigates interference between tasks through sparse, selective information flow. We evaluate MetaAR on the Meta-World benchmark with up to 50 robotic manipulation tasks, where it consistently outperforms strong baselines, achieving 4–8% higher mean success rates than the best-performing methods across the MT10 and MT50 variants.

1. Introduction

Reinforcement Learning has emerged as a powerful framework for sequential decision-making, achieving notable milestones across diverse applications such as vehicular networks [1,2], UAV control [3,4], audio [5,6] and video applications [7]. These advances, largely driven by deep neural networks, have demonstrated the capacity of reinforcement learning to master complex tasks. However, a significant portion of this progress has been confined to isolated, single-task settings, where each task is learned independently. This independence results in substantial inefficiencies in terms of both data usage and computational resources [8,9], especially when scaling to real-world environments that require flexible agents capable of handling multiple related tasks. However, humans naturally generalize across tasks by leveraging shared knowledge and reusing previously acquired skills. Inspired by this, Mul-

titask Reinforcement Learning (MTRL) aims to train a unified policy capable of handling multiple tasks simultaneously [8] as shown in Fig. 1.

By sharing knowledge across tasks, MTRL can improve sample efficiency and enable better generalization [10,11]. However, realizing these benefits introduces new issues. A key issue lies in how to share information effectively: excessive sharing may lead to negative knowledge transfer [12,13], where interference between tasks degrades performance, while insufficient sharing limits the potential gains. Balancing shared and task-specific components is essential, especially when tasks vary in dynamics, goals, or input distributions. Efficient MTRL requires deciding what to share, when, and how to prevent interference.

Recently, a wide range of methods have been proposed to achieve efficient MTRL from different perspectives. A common approach [10,14] is to share a backbone network while using task-specific heads, enabling partial parameter reuse. However, indiscriminate sharing often leads to

* Corresponding authors.

E-mail addresses: panrui805@bupt.edu.cn (R. Pan), haoran.luo@ieee.org (H. Luo), yuanquan@bupt.edu.cn (Q. Yuan), luoguiyang@bupt.edu.cn (G. Luo), jlli@bupt.edu.cn (J. Li), tiesunlong@nus.edu.sg (T. Shen), rui.mao@ntu.edu.sg (R. Mao), cambria@ntu.edu.sg (E. Cambria).

¹ Contributed equally.

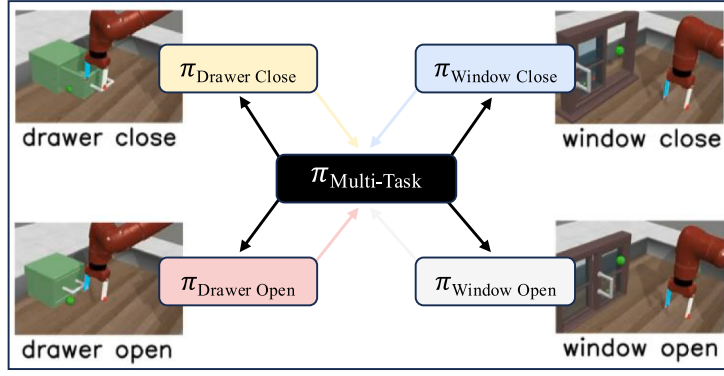


Fig. 1. Illustration of a multitask reinforcement learning framework. The framework consists of four simple robotic arm tasks: opening a drawer, closing a drawer, opening a window, and closing a window. The four task-specific strategies share a unified policy $\pi_{Multitask}$.

negative knowledge transfer, where conflicting task gradients degrade overall performance [15]. Some methods further introduce gradient decomposition techniques [16] or normalization methods [11] to reduce task interference. More recent work explores dynamic routing architectures [13,17], which leverage task representations to generate routing strategies that guide the per-task route selection in order to minimize conflicts. While effective to some extent, these approaches often rely on simplistic task descriptions (e.g., one-hot vectors), limiting the expressiveness and informativeness of the routing decisions.

Others focus on learning latent representations [12,18] to capture shared structures or reusable skills across tasks. Although such representations can improve generalization, they are often treated as auxiliary signals and do not directly influence the policy network’s internal structure. These limitations pose key challenges in designing expressive and adaptable architectures that can accurately model task relationships and enable effective knowledge sharing across diverse tasks.

To overcome these challenges, we propose a novel **Metadata-guided Adaptive Routing (MetaAR)** method for multitask reinforcement learning. MetaAR leverages pretrained language models to embed task metadata and combines them with state features to form expressive task representations. In this work, the metadata of a task refers to its natural-language contextual description provided by the official Meta-World benchmark [19], as shown in Table 1. These fused representations serve not only as informative descriptors but also as semantic priors that guide the model in distinguishing shared and unique aspects across tasks. For example, in “push and close a window” versus “push and open a window”, the task representations capture the shared skill “push” and object “window”, as well as the semantic contrast between “open” and “close”. Then, instead of treating the task representations as auxiliary inputs to a shared policy network, we integrate them directly into each layer of the shared network to dynamically adjust its internal structure. Specifically, the routing network learns to map such similarities and differences in task semantics into distinct routing vectors, which control the activation of module pathways within each layer of the shared policy network. This design enables the model to generate task-specific computation flows that preserve shared features while reducing negative interference among tasks.

To further emphasize task-specific information and alleviate negative knowledge transfer, we introduce a noise-injected Top-K mechanism [20] when configuring the network weights. This mechanism highlights the most relevant routing paths for each task and suppresses low-relevance or potentially interfering ones. Adding noise encourages exploration of diverse paths and prevents premature convergence to sub-optimal routing patterns. MetaAR allows the model to retain the benefits of shared representations while producing task-specific behaviors, effectively balancing generalization and specialization across tasks.

Table 1

Contextual description of tasks for MT10 [19].

Task	Description
press button	Press a button
open door	Open a door with a revolving joint
close drawer	Push and close a drawer
open drawer	Open a drawer
insert peg side	Insert a peg sideways
pick&place	Pick and place a puck to a goal
push	Push the puck to a goal
reach	Reach a goal position
open window	Push and open a window
close window	Push and close a window

In addition, we evaluate our method on the Meta-World benchmark [19], which consists of 50 diverse robotic manipulation and continuous control tasks. We implement our approach alongside several baselines and carry out extensive experiments, including a detailed ablation study. Our method consistently outperforms strong baselines, achieving state-of-the-art performance. Specifically, MetaAR surpasses the best baseline results by 8%, 6%, 6%, and 4% under the MT10-Fixed, MT10-Conditioned, MT50-Fixed, and MT50-Conditioned settings, respectively. The results confirm the effectiveness of our method.

Our main contributions are summarized as follows:

- We propose **MetaAR**, a metadata-guided adaptive routing framework for multitask reinforcement learning. Task metadata is encoded and injected into each layer to control routing in a shared modular policy, rather than serving only as task IDs or auxiliary inputs. To the best of our knowledge, this is the first work that uses semantic metadata to drive layer-wise routing in MTRL.
- We design a noise-injected, differentiable Top-K routing mechanism that performs sparse, task-aware module selection while remaining fully end-to-end trainable. This mitigates negative transfer between tasks and provides an information-theoretic view of how metadata improves task separation through routing.
- We conduct extensive experiments on the Meta-World MT10 and MT50 benchmarks. MetaAR consistently surpasses strong baselines, achieving 4–8% higher mean success rates and improved sample efficiency.

The remainder of this paper is organized as follows. We review related works on multitask reinforcement learning, multitask learning with metadata, and mixture of experts in Section 2. Then we introduce the preliminary concepts of Markov decision process, soft actor-critic, and multitask reinforcement learning in Section 3. Our proposed MetaAR method is detailed in Section 4. The experimental results are

presented and analyzed in Section 5. Finally, we provide conclusions and directions for future research in Section 6.

2. Related work

In this section, we present some studies closely related to our work, divided by multitask reinforcement learning, multitask learning with metadata, and mixture of experts.

2.1. Multitask reinforcement learning

Multitask reinforcement learning addresses the challenge of simultaneously optimizing policies across multiple related tasks by leveraging shared structures and promoting knowledge transfer, thereby improving both sample efficiency and generalization performance [21]. Traditional single-task reinforcement learning methods often suffer from data inefficiency [22], especially when tasks share similar structures or dynamics. By exploiting these commonalities, MTRL methods facilitate the reuse of learned representations, accelerating the overall learning process [8].

Researchers have extensively explored MTRL from various perspectives [8,10,13,17,18,23–25]. A common strategy involves parameter sharing, where tasks share a common network backbone to learn general representations, and individual task-specific heads are used to adapt to each task's unique objective. Representative works include Actor-Mimic [14] and Distral [10], which employ policy distillation to transfer knowledge across tasks. However, naively sharing all knowledge across tasks can lead to negative knowledge transfer [26], where learning on one task interferes with performance on others. To mitigate such task interference, PCGrad [16] projects conflicting gradients onto each other to resolve update conflicts, while PopArt [11] normalizes the value function outputs per task to stabilize learning. Building on the shared backbone paradigm, routing-based architectures have recently emerged as a promising direction to reduce task interference [13,17,27]. These methods learn task-specific computation paths, but most rely on simple task identifiers (e.g., one-hot vectors) to generate routes, limiting the expressiveness and informativeness of the routing decisions. To better model cross-task commonalities, some works have focused on learning interpretable representations that capture shared skills or latent structure across tasks [12,18]. While effective, these representations are typically used only as auxiliary inputs to the policy network, without influencing the internal organization or adaptation of the underlying architecture.

In contrast, our method leverages rich metadata to learn informative and interpretable task representations. Rather than simply using these representations as additional inputs to the policy network, we integrate them directly into the architecture to modulate the internal structure of the shared network. Specifically, the learned task representations are used to dynamically reconfigure the weights of each layer in the backbone network, enabling fine-grained, layer-wise adaptation. This design allows the model to generate task-specific behaviors while preserving the benefits of shared representations, effectively balancing generalization and specialization across tasks.

2.2. Multitask learning with metadata

Multitask learning with metadata studies how auxiliary task-level information, such as task descriptors [28,29], semantic labels [30,31], or structured attributes [32], can be used to encode relationships between tasks and guide model adaptation. In contrast, traditional MTL approaches [33,34] often treat tasks as isolated or only implicitly defined by their training data. Existing work [35–37] has shown that metadata can help decide which tasks should share parameters, how to cluster related tasks, or how to regularize task-specific components, thereby improving generalization across heterogeneous task distributions.

In reinforcement learning, [38] formalizes Contextual Markov Decision Processes (CMDP), where a context variable summarizes task-level information, and [39] shows that natural language descriptions can be

integrated into task definitions in single-task settings. CARE [18] extends this idea to multitask RL by encoding textual metadata and using it as conditioning signals for a shared policy. However, most of these methods treat metadata as additional inputs or high-level context and do not use them to directly organize the internal structure of shared networks. In contrast, inspired by CARE, MetaAR uses task descriptions (for example, “Push and close a window” and “Push and open a window”) to learn task representations that both capture inter-task relationships and dynamically control layer-wise routing in a shared modular policy and Q-function, rather than only enriching the input space.

2.3. Mixture of experts

In recent years, Mixture of Experts (MoE) has shown great potential in reinforcement learning. [40] proposes training a gating function to select different Q-functions (experts) for different tasks. [23] introduces orthogonality constraints among experts to enhance diversity and reduce redundancy. [25] adopts an attention-based MoE architecture, where state representations are generated by multiple expert modules and dynamically weighted through an attention mechanism based on task context. Although MoE architectures offer flexibility, most existing methods still rely on task identifiers or simple task embeddings to determine expert routing, which limits their ability to model complex task relationships. Moreover, maintaining multiple expert networks incurs significant computational overhead. In contrast, our method enhances the model's ability to capture complex inter-task relationships by combining metadata with state information. Instead of designing multiple expert networks, we introduce a single auxiliary network to generate task representations, which are then used to dynamically reconfigure the weights of each layer in a shared backbone network.

3. Preliminaries

In this section, we introduce some prior work related to our research, including Markov decision process, soft actor-critic, and multitask reinforcement learning, as well as some notation definitions. The frequently used mathematical symbols are summarized in Table 2.

3.1. Markov decision process

A Markov Decision Process (MDP) [41] is typically formulated as a tuple $\mathcal{M} = \langle S, \mathcal{A}, \mathcal{P}, R, \rho, \gamma \rangle$, where S denotes the set of states, and $\mathcal{A}$ represents the set of available actions. The transition dynamics are characterized by a probability distribution $\mathcal{P} : S \times \mathcal{A} \rightarrow S$, where $\mathcal{P}(s_{t+1} | s_t, a_t)$ specifies the likelihood of moving to state s_{t+1} given the current state s_t and action a_t . The reward function $R : S \times \mathcal{A} \rightarrow \mathbb{R}$ assigns a scalar reward to each state-action pair. The initial state distribution is given by ρ , and $\gamma \in (0, 1]$ denotes the discount factor that determines the present value of future rewards. A policy π is a mapping from states to a distribution over actions, specifying the behavior of an agent. The expected discounted accumulated rewards under policy π starting from a given state s is described as:

$$J(\pi) = \mathbb{E}_{\pi} \left[\sum_{t=0}^{\infty} \gamma^t r_{t+1} \mid s_0 = s \right], \quad (1)$$

where r_{t+1} is the reward obtained at time step $t + 1$, and s_0 is the initial state. The policy is modeled as a parameterized distribution $\pi_{\theta}(a_t | s_t)$, with θ representing the learnable parameters. The goal is to adjust θ so as to maximize the expected cumulative reward, expressed as $J(\pi_{\theta}) = J(\theta)$.

3.2. Soft actor-critic

Soft Actor-Critic (SAC) [42] is an off-policy reinforcement learning algorithm that augments the standard maximum return objective with an entropy term to encourage exploration. The entropy term promotes

Table 2

Summary of mathematical symbols used in MetaAR.

Symbol	Description
$\mathcal{T}$	Set of tasks in the multitask setting
τ	A specific task sampled from $\mathcal{T}$
c	Contextual metadata associated with task τ
s_t	State of the environment at time step t
a_t	Action taken at time step t
r_t	Reward received at time step t
$\pi_\theta(a s, c)$	Contextual metadata-conditioned policy with parameters θ
$Q_\phi(s, a, c)$	Contextual metadata-conditioned Q-function with parameters ϕ
α	Entropy temperature coefficient
$\tilde{H}$	Target policy entropy
$\mathcal{D}_\tau$	Replay buffer for task τ
z_{s_t}	Encoded state representation
z_c	Encoded context (metadata) representation
$z_{s_t, c}$	Fused task representation
z^l	Unnormalized routing logits for layer l
$\tilde{z}^l$	Gumbel-noised routing logits for layer l
m^l	Top-K binary mask for routing at layer l
p^l	Masked routing probability matrix for layer l
x_j^l	Input to the j -th module in layer l
W_j^l	Weight matrix of the j -th module in layer l
g^l	Gumbel noise vector
μ, σ	Action distribution parameters (mean and std)
L	Number of module layers in the base policy network
n	Number of modules per layer
k	Top-K sparsity value in routing
β	Scale factor for Gumbel noise
R^l	the routing decision at layer l
$H(\cdot)$	the Shannon entropy
$I(\cdot)$	the mutual information

stochasticity in the policy by favoring actions with higher entropy. The policy objective for SAC is formulated as:

$$J_{\text{SAC}}(\pi) = \mathbb{E}_{\zeta \sim \pi} \left[\sum_{t=0}^{\infty} \gamma^t (r_t + \alpha \mathcal{H}(\pi(\cdot | s_t))) \right], \quad (2)$$

where $\zeta = (s_0, a_0, r_1, \dots)$ is a trajectory, $\mathcal{H}(\pi(\cdot | s_t)) = -\mathbb{E}_{a_t \sim \pi} [\log \pi(a_t | s_t)]$ denotes the entropy of the policy at state s_t , $\gamma \in (0, 1]$ is the discount factor, and $\alpha > 0$ is a temperature parameter that controls the trade-off between reward maximization and entropy. SAC optimizes this objective by alternately updating the policy π_θ and the soft Q-function Q_ϕ . The policy is represented as a parameterized distribution $\pi_\theta(a | s)$ and the Q-function is parameterized by ϕ . The objectives for the policy and Q-function are given by:

$$J_\pi(\theta) = \mathbb{E}_{s \sim \mathcal{D}} \left[\mathbb{E}_{a \sim \pi_\theta} [\alpha \log \pi_\theta(a | s) - Q_\phi(s, a)] \right], \quad (3)$$

$$J_Q(\phi) = \mathbb{E}_{(s, a, r, s') \sim \mathcal{D}} [(Q_\phi(s, a) - (r + \gamma \mathbb{E}_{a' \sim \pi_\theta} [Q_\phi(s', a') - \alpha \log \pi_\theta(a' | s')]))^2], \quad (4)$$

where Q_ϕ is a target Q-network with delayed parameters, and $\mathcal{D}$ is a replay buffer storing experience tuples. The temperature parameter α can also be learned by minimizing the following objective to match the policy entropy to a target value $\tilde{H}$:

$$J(\alpha) = \mathbb{E}_{a \sim \pi} [-\alpha (\log \pi(a | s) + \tilde{H})]. \quad (5)$$

3.3. Multitask reinforcement learning

In the setting of multitask reinforcement learning, an agent is required to interact with a collection of tasks $\tau \in \mathcal{T}$, where each task τ corresponds to a distinct MDP denoted by $\mathcal{M}^\tau = (S^\tau, \mathcal{A}^\tau, \mathcal{P}^\tau, \mathcal{R}^\tau, \rho^\tau, \gamma^\tau)$. We denote the shared policy by $\pi_\theta(a_t | s_t)$, where θ represents the learnable parameters. The objective is to optimize a single policy that achieves high expected return across the full distribution of tasks. These tasks are assumed to be drawn from a task distribution $p(\mathcal{T})$, and each task may vary in MDP components. In our work, we describe the MTRL problem

using the Block Contextual Markov Decision Process (BC-MDP) framework [18]. A BC-MDP is defined as a tuple $\langle S, \mathcal{A}, \gamma, \mathcal{M}', C \rangle$, where S is the state space, $\mathcal{A}$ is the action space shared across tasks, and C denotes the contextual metadata space. The function $\mathcal{M}'$ acts as a mapping from contextual metadata $c \in C$ to task-specific MDP components, $\mathcal{M}'(c) = \{S_c, \mathcal{P}_c, \mathcal{R}_c, \rho_c\}$, where S_c represents the task's state space, $\mathcal{P}_c$ the transition dynamics, $\mathcal{R}_c$ the reward function, and ρ_c the initial state distribution corresponding to contextual metadata c . And each task τ and its associated MDP parameters are identified and indexed by its contextual metadata c .

To adapt SAC to the multitask setting, we condition the policy and value functions on a task contextual metadata $c \in C$, where each c defines a specific MDP. The shared policy is denoted by $\pi_\theta(a | s, c)$, and the objective is to maximize the expected entropy-augmented return across tasks:

$$J_{\text{MT-SAC}}(\pi) = \mathbb{E}_{c \sim q(C)} \left[\mathbb{E}_{\zeta \sim \pi_\theta(\cdot | \cdot, c)} \left[\sum_{t=0}^{\infty} \gamma^t (r_t^c + \alpha \mathcal{H}(\pi_\theta(\cdot | s_t, c))) \right] \right], \quad (6)$$

where $q(C)$ represents the distribution of contextual metadata c . The Q-function and value targets are also conditioned on c , and all components are trained using transitions labeled with their corresponding task metadata.

4. Methodology

We propose **Metadata-guided Adaptive Routing (MetaAR)**, a novel framework for multitask reinforcement learning that enables dynamic, task-specific modulation of shared policy networks. The key idea is to leverage metadata (such as natural language task descriptions) along with task state observations to produce adaptive routing weights that reconfigure shared network paths. To improve both task specialization and exploration, we inject Gumbel noise [43] into a differentiable Top-K selection mechanism, which highlights critical computation paths for each task. The architecture of MetaAR is illustrated in Fig. 2. It consists of two main components: a Base Policy Network, which divides all parameters into separate modules to learn different knowledge, and a Routing Network, which determines how these modules are combined. These two networks work together to generate dynamic, task-conditioned network behavior.

4.1. Routing network

In this subsection, we describe the design of the routing network in MetaAR, which takes the fused task representation as input and produces layer-wise routing decisions to modulate the shared modular backbone. The routing network serves as the key component that translates semantic information about each task into sparse and task-aware computation paths. Concretely, we first introduce how task representations are constructed from state observations and metadata, and then present the noise-injected Top-K routing mechanism that enables selective information flow and reduces interference across tasks.

4.1.1. Task representation with metadata

At each time step t , our MetaAR framework receives two types of inputs: the current task state s_t and the metadata of the task τ , typically provided as a contextual description c (e.g., “push and close a drawer”). We first encode the state s_t via a single-layer MLP:

$$z_{s_t} = f(s_t) \in \mathbb{R}^{D_s}. \quad (7)$$

To encode the metadata c of task τ , which is given in natural language form, we follow the setting in CARE [18] and employ RoBERTa [44], a robustly optimized transformer-based language model, to obtain a fixed-dimensional embedding. This design choice is intentional: our goal in this work is to isolate the effect of metadata-guided adaptive routing on multitask RL, rather than to compare different language models. By using the same RoBERTa-based encoder as CARE, all methods operate in a

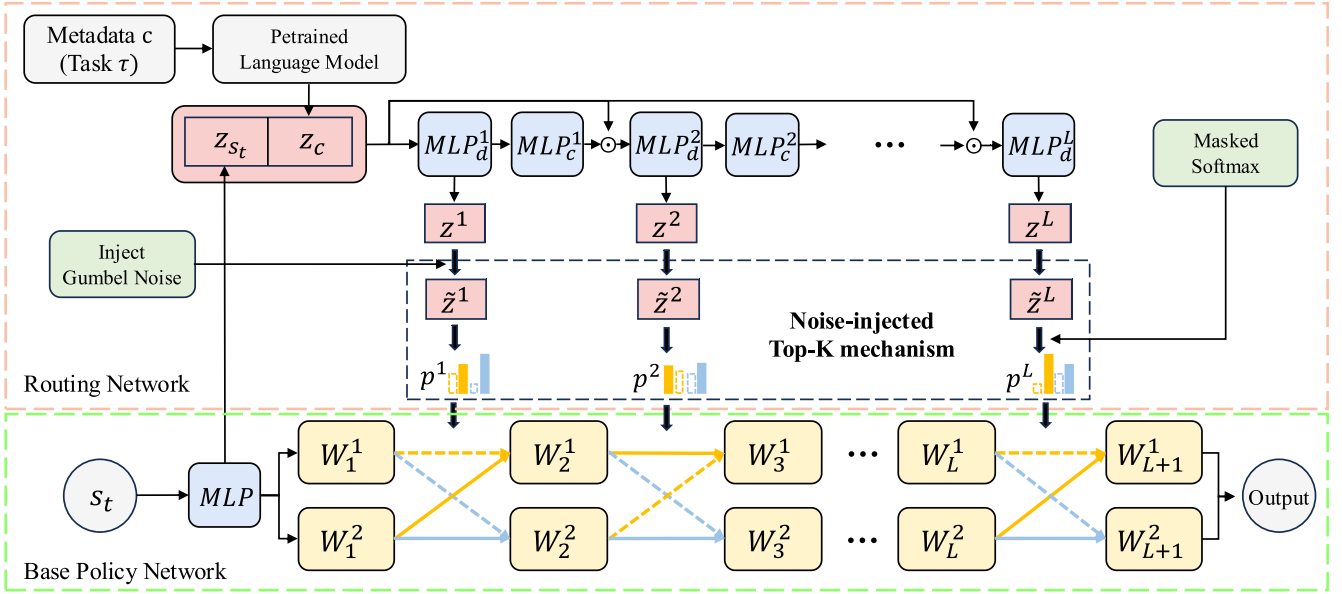


Fig. 2. Overview of the Metadata-guided Adaptive Routing (MetaAR) architecture. The framework consists of two main components: a Routing Network and a Base Policy Network. The Routing Network takes the current state and task metadata as input and produces layer-wise routing probabilities, while the Base Policy Network is organized into modular layers whose inter-module connections are dynamically controlled by these routing decisions. Bars with the same color correspond to the same target module across layers, and hollow bars indicate connections that are masked out by the Top-K operation, which enforces sparse, task-specific inter-layer connections in the base policy network.

comparable metadata representation space and the performance differences in Section 5 can be primarily attributed to the routing mechanism itself. We extract the contextualized embedding from the final [CLS] token position and apply a linear projection h :

$$z_c = h(\text{RoBERTa}(c)) \in \mathbb{R}^{D_c}. \quad (8)$$

We then concatenate the two embeddings to form the unified task representation:

$$z_{s_t, c} = [z_{s_t}; z_c] \in \mathbb{R}^{D_s + D_c}. \quad (9)$$

This fused representation $z_{s_t, c}$ serves as the input to the routing network, which produces task-conditioned routing weights for adaptive path selection.

4.1.2. Adaptive routing with noise-injected top-K

Our routing network is designed to generate task-conditioned soft weights that modulate the execution paths within the base policy network. It consists of multiple MLP layers and is conditioned on the fused task representation $z_{s_t, c}$. The routing network computes layer-wise control vectors that guide module selection within each layer of the base policy network through a differentiable Top-K selection process enhanced by Gumbel noise.

Routing Vector Computation. Suppose the base policy network has $L + 1$ module layers, and each module layer contains n modules, represented as $W_j^l, l = 1, 2, \dots, L + 1; j = 1, 2, \dots, n$ as shown in Fig. 2. Then for each current task state s_t and metadata c , the routing network must generate L vectors $\{z^1, z^2, \dots, z^L\}$, where each $z^l \in \mathbb{R}^{n^2}$ represents the routing vector of the l -th layer for the current task.

Each routing vector z^l is computed from the fused task representation $z_{s_t, c}$ via feedforward layers. When $l = 1$, the routing vector $z^{l=1}$ can be expressed as:

$$z^1 = MLP_d^1(\text{ReLU}(z_{s_t, c})), \quad (10)$$

where MLP_d^1 is a fully connected layer with a dimension of $\mathbb{R}^{n^2 \times (D_s + D_c)}$. When $l = 2, 3, \dots, L$, the routing vector z^l can be expressed as:

$$z^l = MLP_d^l(\text{ReLU}(MLP_c^l(z^{l-1}) \odot z_{s_t, c})), l = 2, 3, \dots, L, \quad (11)$$

where MLP_c^l is a fully connected layer with dimension $\mathbb{R}^{(D_s + D_c) \times n^2}$, which converts the routing vector z^l into an embedding of the same dimension as the fused task representation $z_{s_t, c}$. To allow layer-specific adaptation to the current task and environment state, we re-inject the fused task representation $z_{s_t, c}$ at each layer of the routing network. This enables each layer to make independent, context-aware routing decisions, improving path diversity and preventing over-reliance on earlier routing outputs. Specifically, we perform the element-wise multiplication of the converted embedding with the initial fusion task representation $z_{s_t, c}$ and then input it into the feedforward layer MLP_d^l .

Noise-injected Top-K Routing. To enhance task-specific module selection and encourage exploration over diverse routing paths, we adopt a Top-K routing strategy augmented with Gumbel noise and Masked Softmax. This design activates only the most relevant connections for each task while maintaining differentiability, enabling stable end-to-end optimization. By promoting selective information flow and task-aware sparsity, the routing mechanism effectively mitigates negative knowledge transfer across tasks, ensuring that task-specific knowledge is preserved without interference from unrelated tasks.

Given the unnormalized routing vector $z^l \in \mathbb{R}^{n^2}$ for layer l , we first inject Gumbel noise to promote stochastic exploration during training:

$$z'^l = z^l + \beta \cdot g^l, \quad g^l \sim \text{Gumbel}(0, 1), \quad (12)$$

where g^l is a Gumbel noise vector sampled independently for each task and layer, and β is a hyperparameter controlling the noise scale. This noise injection allows the model to explore alternative routing configurations during training. Let $z'^l \in \mathbb{R}^{n^2}$ be reshaped into a matrix form $\tilde{z}^l \in \mathbb{R}^{n \times n}$, where each element $\tilde{z}_{i,j}^l$ represents the unnormalized routing score of the connection from the j -th module in layer l to the i -th module in layer $l + 1$.

We then apply a Top-K masking operation to identify the k most important connections for each target module. Let $m^l \in \{0, 1\}^{n \times n}$ be the binary mask defined as:

$$m_{i,j}^l = \text{TopKMask}(\tilde{z}_i^l, k)_j = \begin{cases} 1, & \text{if } \tilde{z}_{i,j}^l \text{ is in top-}k \text{ of row } \tilde{z}_i^l. \\ 0, & \text{otherwise.} \end{cases} \quad (13)$$

To obtain a valid probabilistic routing distribution across modules while respecting the sparsity constraint, we apply the Masked Softmax over the masked entries to normalize each routing vector:

$$p_{i,j}^l = \text{MaskedSoftmax}(\bar{z}_i^l, m_i^l)_j = \frac{\exp(\bar{z}_{i,j}^l) \cdot m_{i,j}^l}{\sum_{v=1}^n \exp(\bar{z}_{i,v}^l) \cdot m_{i,v}^l}, \quad (14)$$

the resulting $p_{i,j}^l$ defines a sparse, differentiable routing probability from module j in layer l to module i in layer $l+1$. This formulation ensures that each module in the next layer receives input only from the top- k contributors, while maintaining a normalized probability distribution over them.

By injecting noise during training and enforcing hard Top-K sparsity, this mechanism encourages the model to explore diverse and potentially optimal module combinations, while preventing premature convergence to suboptimal or overly rigid routing patterns.

4.2. Base policy network

The base policy network is a shared multi-layer modular architecture used to generate task-specific actions. It is composed of $L+1$ layers, each containing n parameterized modules. These modules are denoted as $\{W_j^l\}_{l=1, L+1, j=1, n}$, where W_j^l represents the parameters of the j -th module in the l -th layer. Each module independently transforms its input feature through a nonlinear mapping, typically a linear layer followed by a ReLU activation.

At each layer l , the task-specific routing probabilities $p^l \in \mathbb{R}^{n \times n}$, computed by the routing network (as defined in the previous section), are used to control how the outputs of the n modules in layer l are routed to the n modules in layer $l+1$. This allows each downstream module to receive a sparse, weighted combination of outputs from the previous layer, modulated by the task-conditioned mask.

Let $x_j^l \in \mathbb{R}^d$ denote the input of module j in layer l . Then, the input to module i in layer $l+1$ is computed as:

$$x_i^{l+1} = \sum_{j=1}^n p_{i,j}^l \cdot \text{ReLU}(W_j^l x_j^l). \quad (15)$$

After the final modular layer $L+1$, we aggregate the outputs of the n modules to produce the final policy output. Let $x_j^{L+1} \in \mathbb{R}^d$ be the input of the j -th module in the last layer, and $W_j^{L+1} \in \mathbb{R}^{o \times d}$ be its corresponding output projection weights, where o is the output dimension. We first compute the final output vector O as the weighted sum of the transformed module outputs:

$$O = \sum_{j=1}^n W_j^{L+1} x_j^{L+1}, \quad O \in \mathbb{R}^o. \quad (16)$$

Following standard SAC implementations [42], we set $o = 2d_a$, where d_a denotes the action dimension. The policy parameters are then obtained by splitting O into two halves:

$$\mu = O_{1:d_a}, \quad \log \sigma = O_{d_a+1:2d_a}, \quad (17)$$

where μ and $\log \sigma$ represent the mean and log standard deviation of the action.

While the previous sections focus on the policy network architecture, we also apply MetaAR to the Q-function. Importantly, the base policy network and routing network used for the Q-function are implemented independently from those used in the policy network. Specifically, for each Q-network, we instantiate an independent routing network and modular base network with the same layer configuration and metadata conditioning as the policy network, but with separate parameters and optimizers. The policy network and the Q-function are trained jointly in the standard SAC loop using a shared replay buffer: at each update step, we first update the Q-network by minimizing the Bellman error, and then update the policy network while keeping the Q-network fixed. No additional pre-training or sequential optimization is used. It follows prior modular MTRL methods [13,17], where the Q-function shares the

same modular routing structure as the policy network but does not share weights.

4.3. Training and optimization

MetaAR enables the model to dynamically reconfigure computation pathways for each task by the layer-wise sparse aggregation mechanism. By routing through only a subset of modules per layer, the model can share relevant knowledge across related tasks while reducing interference from unrelated ones. This structure also improves parameter efficiency, as all modules are shared across tasks, yet the actual execution paths are selectively activated via task-conditioned routing. Our method is trained under the Multitask Soft Actor-Critic (MT-SAC) [42] framework, which extends the entropy-augmented reinforcement learning objective to the multitask setting.

4.3.1. Multitask SAC objective

As described in Section 3, we model each task τ as a distinct MDP $\mathcal{M}^\tau$, associated with a corresponding task context $c \in \mathcal{C}$. To support generalization and task-specific behavior, we condition both the actor and critic networks on the fused representation $z_{s_t, c}$ derived from the state s_t and task contextual description c .

Following the multitask SAC formulation, the objective is to learn a shared policy that maximizes the expected cumulative reward across tasks as Eq. (6):

$$J_{\text{MT-SAC}}(\pi) = \mathbb{E}_{c \sim q(\mathcal{C})} \left[\mathbb{E}_{\xi \sim \pi_\theta(\cdot | \cdot, c)} \left[\sum_{t=0}^{\infty} \gamma^t (r_t^c + \alpha \mathcal{H}(\pi_\theta(\cdot | s_t, c))) \right] \right]. \quad (18)$$

4.3.2. Context-conditioned loss functions

The policy π_θ and Q-function Q_ϕ are parameterized using separate instances of the MetaAR architecture, each equipped with its own routing network and modular base layers. Their losses are defined analogously to standard SAC, extended with task context conditioning.

$$\mathcal{L}_\pi(\theta) = \mathbb{E}_{(s,c) \sim D} \left[\mathbb{E}_{a \sim \pi_\theta(\cdot | s, c)} [\alpha \log \pi_\theta(a | s, c) - Q_\phi(s, a, c)] \right], \quad (19)$$

$$\mathcal{L}_Q(\phi) = \mathbb{E}_{(s,a,r,s',c) \sim D} [(Q_\phi(s, a, c) - (r + \gamma \mathbb{E}_{a' \sim \pi_\theta(\cdot | s', c)} [Q_\phi(s', a', c) - \alpha \log \pi_\theta(a' | s', c)])^2], \quad (20)$$

where Q_ϕ denotes the target Q-network with delayed parameters, and D is a replay buffer storing experience tuples. The temperature parameter α can also be learned as Eq. (5), extended with task context conditioning:

$$\mathcal{L}(\alpha) = \mathbb{E}_{a \sim \pi_\theta(\cdot | s, c)} [-\alpha (\log \pi_\theta(a | s, c) + \bar{H})], \quad (21)$$

where $\bar{H}$ is the target entropy.

4.3.3. Training procedure

During each training step, we sample a task τ and its corresponding metadata c . A batch of experience tuples (s, a, r, s') is retrieved from the task-specific buffer D_τ , and the fused representation $z_{s, c}$ is computed. This representation is used to generate routing weights $\{p^l\}_{l=1}^L$ via the routing network, and guide the modular forward pass through either the actor or critic network. The full update procedure is described in Algorithm 1.

4.3.4. Theoretical analysis

We next provide a brief information-theoretic view [45] that helps explain why metadata-guided routing tends to reduce task interference. Let $\tau \in \mathcal{T}$ denote the task identity, s the state observation, $c \in \mathcal{C}$ the contextual metadata, and R^l the routing decision at layer l (e.g., the selected modules). Intuitively, task interference is aggravated when different tasks induce similar routing distributions, i.e., when $P(R^l | \tau)$ and $P(R^l | \tau')$ are almost indistinguishable for $\tau \neq \tau'$. In such cases, gradients from different tasks are repeatedly aggregated on the same subset of modules, leading to conflicting updates.

Algorithm 1: Multitask SAC training with MetaAR.

Input: Task set $\mathcal{T}$, replay buffers $\{D_\tau\}_{\tau \in \mathcal{T}}$, number of modules n , number of layers L , temperature α , Top-K value k , noise scale β

Output: Trained policy π_θ and Q-function Q_ϕ with task-conditioned routing

```

1 while not converged do
2   Sample task  $\tau \sim \mathcal{T}$  and corresponding metadata context  $c$ ;
3   Sample a minibatch  $\{(s, a, r, s')\}$  from buffer  $D_\tau$ ;
   // Construct fused task representation
4    $z_{s,c} \leftarrow \text{Concat}(MLP(s), h(\text{RoBERTa}(c)))$ ;
   // Compute routing probabilities
5   for  $l = 1$  to  $L$  do
6     Compute routing vector  $z^l$  using  $MLP_d^l$  and  $MLP_c^l$ ;
7     Add Gumbel noise:  $z^{l'} = z^l + \beta \cdot g^l$ ;
8     Reshape  $z^{l'}$  into matrix  $\tilde{z}^l$ ;
9     Apply Top-K mask:  $m^l = \text{TopKMask}(\tilde{z}^l, k)$ ;
10    Compute masked softmax:  $p^l = \text{MaskedSoftmax}(\tilde{z}^l, m^l)$ ;
   // Forward pass through MetaAR
11  Use  $\{p^l\}_{l=1}^L$  to compute actor output  $(\mu, \sigma)$  and sample
    $a' \sim \pi_\theta(\cdot | s', c)$ ;
12  Compute critic outputs  $Q_\phi(s, a, c)$  and  $Q_\phi(s', a', c)$ ;
   // Update losses
13   $\mathcal{L}_Q \leftarrow$  TD error as Eq. (20);
15   $\mathcal{L}_\pi \leftarrow$  actor loss as Eq. (19);
17   $\mathcal{L}_\alpha \leftarrow$  temperature loss as Eq. (21);
   // Gradient descent updates
19  Update critic parameters  $\phi$  using  $\nabla_\phi \mathcal{L}_Q$ ;
20  Update actor parameters  $\theta$  using  $\nabla_\theta \mathcal{L}_\pi$ ;
21  Update temperature  $\alpha$  using  $\nabla_\alpha \mathcal{L}_\alpha$ ;
22  Soft update target network;

```

A convenient quantity to characterize how well routing separates tasks is the mutual information between the task and the routing decision:

$$I(\tau; R^l) = H(R^l) - H(R^l | \tau), \quad (22)$$

where $H(\cdot)$ is the Shannon entropy [46]. A larger $I(\tau; R^l)$ means that knowing the task greatly reduces the uncertainty about which routing pattern is used, implying more task-specific module usage and less expected overlap across tasks. Conversely, if $I(\tau; R^l)$ is small, routing decisions are nearly independent of the task and the resulting module usage is highly shared, which increases interference.

The role of metadata can be understood through the data processing inequality. In a metadata-agnostic setting, routing is a function of the state only, $R^l = \text{Route}^l(s; \theta)$, and thus

$$I(\tau; R^l) \leq I(\tau; s). \quad (23)$$

In our metadata-guided setting, routing depends on both state and metadata, $R^l = \text{Route}^l(s, c; \theta)$. Since the contextual metadata c is informative about the task (e.g., it explicitly encodes semantics such as “open”, “close”, or “push”), the pair (s, c) contains at least as much information about τ as s alone:

$$I(\tau; (s, c)) \geq I(\tau; s), \quad (24)$$

with strict inequality whenever c provides additional task information beyond the state. Consequently, the class of routing functions that can exploit (s, c) has a strictly larger information capacity to encode task identities than the class that only observes s . At an ideal optimum (assuming sufficient expressiveness and effective training), a metadata-guided router can therefore achieve

$$I_{\text{meta}}(\tau; R^l) \geq I_{\text{no-meta}}(\tau; R^l), \quad (25)$$

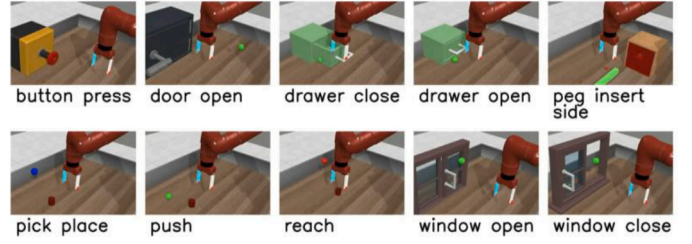


Fig. 3. MT10 Environment.

which translates into a lower expected overlap of routing patterns across tasks and thus reduced interference.

5. Experiment

To demonstrate the effectiveness and generalization ability of our proposed method, MetaAR, we conduct comprehensive evaluations on the Meta-World [19] benchmark, a suite of diverse robotic manipulation tasks. This section introduces the experimental environment, the baseline methods, and the implementation details. We then compare MetaAR with several representative approaches, followed by ablation studies to analyze the contribution of each component in our framework.

5.1. Experimental setup

In this subsection, we detail the experimental setup used to evaluate MetaAR on the Meta-World benchmark. We first describe the environment configuration and task settings, and then introduce the baseline methods for comparison, the evaluation metrics, and the implementation details and hyperparameters shared across all methods. This structured setup allows for a fair and systematic comparison between MetaAR and existing approaches under both MT10 and MT50, fixed-goal and goal-conditioned scenarios.

5.1.1. Environment

We evaluate our proposed method, MetaAR, on the Meta-World benchmark, a widely adopted multitask robotic manipulation environment built on the MuJoCo [47] physics engine. Meta-World features 50 distinct manipulation tasks involving a robotic arm interacting with objects such as drawers, buttons, and windows. Each task requires goal-directed control under continuous action spaces, and all tasks share a consistent observation space dimensionality.

To benchmark multitask performance, we consider two standard settings: MT10 and MT50, which comprise 10 and 50 tasks, respectively, trained jointly within a unified policy. These benchmark variants introduce challenges in generalization, transferability, and interference across diverse tasks. Following the improved protocol introduced in [13], we experiment under both fixed-goal and goal-conditioned variants:

- MT10-Fixed and MT50-Fixed refer to scenarios where each task is associated with a predefined static target.
- MT10-Conditioned and MT50-Conditioned extend these settings to randomly sampled target configurations at each episode, offering greater diversity and scalability for evaluating general-purpose multitask agents.

As illustrated in Fig. 3, the benchmark contains representative tasks such as “button press”, “door open”, “drawer open/close”, “window open/close”, “pick and place”, and “push”, which collectively evaluate an agent’s ability to learn a unified policy that generalizes across diverse manipulation behaviors.

Table 3

Common hyperparameter values across all evaluated methods.

Hyperparameter	Values
network architecture	feedforward network
batch size	$128 \times \text{number of tasks}$
policy initialization	standard Gaussian
# of samples / # of train steps per iteration	1 env step / 1 training step
activation function	ReLU
optimizer	Adam
policy learning rate	$3e-4$
beta for Adam optimizer (policy)	(0.9, 0.999)
Q-function learning rate	$3e-4$
beta for Adam optimizer (Q function)	(0.9, 0.999)
exploration steps	$2000 \times \text{number of tasks}$
discount factor	0.99
episode length	200
reward scale	0.1
replay buffer size	$1e6$ (MT10) / $1e7$ (MT50)
temperature	learned and disentangled with tasks

Table 4

Additional hyperparameter values specific to each evaluated method.

Model	Hyperparameter	Values
MT-SAC	network size	[400, 400, 400, 400, 400]
MT-MH-SAC	network size	[400, 400, 400, 400, 400]
	network architecture	multi-head (1 head/task)
PCGrad	network size	[400, 400, 400, 400, 400]
SM	state representation size	[400, 400]
	task representation size	[400]
	number of module layers	4
	number of modules/layer	4
	module size	128
	routing size	[256, 256, 256, 256, 256]
CARE	state representation size	[400, 400, 400]
	task representation size	[128, 128]
	number of encoders	6
	encoder gating size	[128, 128]
	encoder network size	[128, 128]
PaCo	param-set size	[400, 400, 400]
	number pf param-sets	5
	extreme loss threshold	$3e3$
MetaAR (ours)	state representation size	[400, 400]
	task representation size	[400]
	number of module layers	4
	number of modules/layer	4
	module size	128
	routing size	[256, 256, 256, 256, 256]
	Top-K	Top-2
	noise scale	decays from 1.0

5.1.2. Baselines

To evaluate the effectiveness of our proposed **MetaAR** framework, we compare it against a broad range of representative baselines. These include standard multitask SAC variants, gradient-based optimization strategies, modular architectures, and context-aware learning approaches. Below is a detailed introduction to the methods considered:

- **Multitask SAC (MT-SAC)** [19]. A standard baseline where a single SAC policy is shared across all tasks. No task-specific information is explicitly provided, and the model learns a unified policy via parameter sharing.
- **Multitask Multi-header SAC (MT-MH-SAC)** [19]. A shared network backbone is used for feature extraction, followed by separate task-specific heads for each task. This allows each task to maintain its own output layer, alleviating task interference while maintaining shared representations.

- **PCGrad** [16]. A gradient surgery technique that addresses negative gradient interference by projecting each task's gradient onto the normal plane of any conflicting gradient. It improves multitask optimization without modifying the network structure.
- **Soft Modularization (SM)** [13]. A dynamic routing architecture that employs a routing network to softly assign tasks to different module paths within a shared backbone. This enables per-task network reconfiguration and improves robustness to task conflicts.
- **CARE** [18]. A metadata-guided multitask reinforcement learning method that utilizes natural language descriptions as contextual inputs. These metadata are encoded using pretrained language models and serve as input to a shared policy network, enhancing generalization.
- **PaCo** [24]. A parameter-compositional method for MTRL that learns a task-agnostic parameter set and task-specific compositional vectors. Policies for each task are represented by linear combinations in parameter space, allowing flexible sharing and improved training stability through loss masking and compositional vector reset.

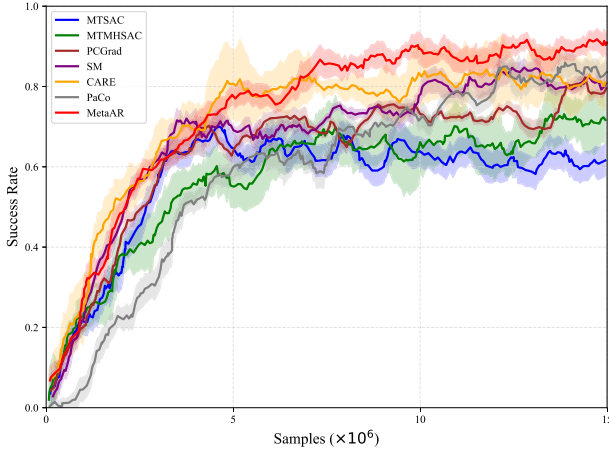
5.1.3. Evaluation metrics

Following the standard practice in Meta-World [19], we adopt the **Mean Success Rate (Mean SR)** as our primary evaluation criterion. Specifically, the success rate for each task is computed based on whether the manipulated object reaches its designated goal configuration within a predefined Euclidean distance threshold ϵ , i.e., $\|o - g\|_2 < \epsilon$, where a commonly used example is $\epsilon = 5$ cm in the official Meta-World implementation [19]. The Mean SR is then calculated as the average success rate across all tasks. For exact definitions of task-specific distance thresholds and goal conditions, we refer readers to the original Meta-World benchmark documentation.

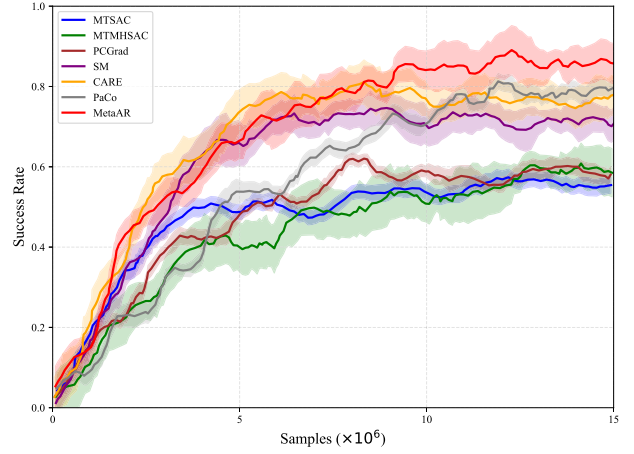
5.1.4. Implementation details

In this section, we first detail the hyperparameter configurations used in our experimental comparisons. All experiments are implemented in Python 3.7.13 using the PyTorch 1.11.0 toolkit with CUDA 11.3, and are conducted on two Linux machines: a workstation equipped with an Intel Core i9-13900K CPU, 128 GB of RAM, and two NVIDIA RTX 4090 GPUs (24 GB each), and a server equipped with two Intel Xeon Gold 6248R CPUs, 256 GB of RAM, and eight NVIDIA Tesla V100 GPUs (32 GB each), both running Ubuntu 20.04 LTS. **Table 3** lists the set of parameters that are consistently applied across all evaluated methods. And **Table 4** summarizes the additional hyperparameters specific to each baseline and our method, MetaAR. In addition, we apply two previously proposed training techniques across all methods: loss maskout of extreme tasks, as introduced in [18,24], and multitask loss rescaling, proposed in [13]. For detailed implementation and technical specifics, please refer to the original papers.

Additionally, we evaluate all baselines and MetaAR on the Meta-World-V2 benchmark [19]. For the MT10 setting, we train each task for 1.5 million steps. Specifically, we train 15 million steps on the MT10 setting. For the MT50 setting, we reduce the number of environment steps sampled per task during training, considering that the overall training time increases with the number of tasks. Moreover, effective knowledge sharing across tasks can improve sample efficiency and accelerate convergence. Specifically, we train 50 million steps (1 million steps for each task) for the MT50 setting. In all experiments, each method is trained using 3 random seeds. For evaluation, we sample 100 episodes per task per seed and report the final performance by computing the Mean SR across all tasks. To ensure fairness, all baselines are re-implemented within the Meta-World-V2 environment and evaluated under the same hardware settings.



(a) MT10-Fixed



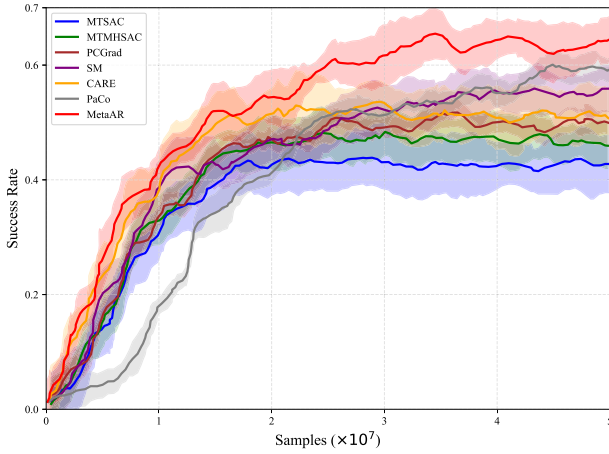
(b) MT10-Conditioned

Fig. 4. Training curves of different methods on MT10.

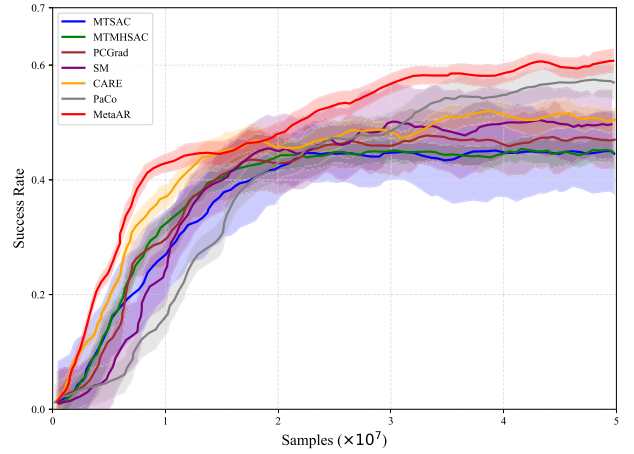
Table 5

Comparison of Mean SR on Meta-World environment under MT10-Fixed, MT10-Conditioned, MT50-Fixed, and MT50-Conditioned settings.

Methods	MT10-Fixed	MT10-Conditioned	MT50-Fixed	MT50-Conditioned
MT-SAC	0.68 ± 0.06	0.55 ± 0.02	0.42 ± 0.05	0.44 ± 0.08
MT-MH-SAC	0.72 ± 0.08	0.59 ± 0.06	0.46 ± 0.04	0.44 ± 0.02
PCGrad	0.78 ± 0.03	0.58 ± 0.02	0.50 ± 0.02	0.46 ± 0.05
SM	0.80 ± 0.04	0.73 ± 0.04	0.55 ± 0.04	0.49 ± 0.04
CARE	0.82 ± 0.08	0.76 ± 0.05	0.52 ± 0.06	0.50 ± 0.02
PaCo	0.80 ± 0.05	0.78 ± 0.02	0.58 ± 0.02	0.56 ± 0.03
MetaAR (Ours)	0.90 ± 0.04	0.84 ± 0.05	0.64 ± 0.04	0.60 ± 0.02



(a) MT50-Fixed



(b) MT50-Conditioned

Fig. 5. Training curves of different methods on MT50.

5.2. Experimental results

To visualize the training dynamics, we plot the Mean SR along with its variance across different stages of training, as shown in Figs. 4 and 5. And the final performance evaluation results are summarized in Table 5.

MT10-Fixed and MT10-Conditioned. As shown in Fig. 4, MetaAR demonstrates superior performance in both the MT10-Fixed and MT10-Conditioned settings. In the early stage of training (within the first 5M steps), MetaAR and CARE exhibit a faster performance gain compared to other methods. We attribute this to the incorporation of metadata as a form of prior knowledge, which facilitates the capture of shared representations. This strong representational constraint enables transferable

features across different tasks to converge more rapidly, thereby improving sample efficiency in the early stages of training. And as training converges, MetaAR consistently achieves the highest mean success rate. As reported in the second and third columns of Table 5, in the MT10-Fixed setting, MetaAR attains a Mean SR that is 8% higher than the best-performing baseline, CARE. Additionally, in the MT10-Conditioned setting, MetaAR surpasses the best-performing baseline, PaCo, by 6%. These improvements are substantial and highlight the effectiveness of MetaAR.

MT50-Fixed and MT50-Conditioned. As the number of tasks increases, joint training becomes more challenging. As shown in Fig. 5, MetaAR achieves impressive performance under both MT50 settings,

converging the fastest and attaining the highest performance after convergence. As reported in the fourth and fifth columns of Table 5, MetaAR outperforms the best baseline methods by 6% and 4% in the two MT50 settings, respectively. With the growing number of tasks, inter-task interference becomes more severe. By incorporating metadata and dynamically adjusting the routing structure, MetaAR provides more appropriate and accurate routing for each task, thereby mitigating inter-task interference and achieving better generalization.

Details on Computational Resources. To provide a clearer picture of the computational cost of our method, we report the training time of MetaAR and representative baselines on the MT10 benchmark. All experiments are run with the same training budget of 15 million steps on a server equipped with eight NVIDIA Tesla V100 GPUs (32 GB each), as described in Section 5.1.4. Under this setting, MT-SAC is the fastest and requires approximately 552 GPU hours. The soft-modularization baseline SM takes about 636 GPU hours. Our MetaAR further introduces noise-injected Top-K routing, and completes MT10 training in approximately 604 GPU hours. Interestingly, MetaAR is slightly more efficient than SM in terms of GPU hours, even though it employs an additional routing mechanism. This is because SM relies on dense, soft routing, where all modules in each layer are active for every forward pass, whereas MetaAR uses sparse Top-K routing and only activates a small subset of modules per layer for each task. As a result, the effective number of module evaluations per step is reduced, and the overhead of sampling Gumbel noise and computing the Top-K mask is negligible compared to the cost of full-module forward passes.

Overall, on MT10, MetaAR achieves the best final performance and sample efficiency among all compared methods (see Fig. 4 and Table 5). While MetaAR incurs a moderate increase in computational cost compared to the simplest baseline MT-SAC, this overhead is practically manageable and is offset by substantial gains in multitask performance and data efficiency. At the same time, MetaAR remains more efficient than SM in terms of GPU hours, showing that the metadata-guided sparse routing design can improve both effectiveness and efficiency in the multitask setting.

5.3. Visualization analysis

To gain deeper insights into the MetaAR method, we visualize the network trained with MetaAR under the MT10-Conditioned setting.

t-SNE Visualization. We evaluate the final policy by running each task three times and collecting routing probability samples. For each task, we gather 1000 samples and then apply t-SNE [48] for dimensionality reduction and visualization. The results are shown in Fig. 6. The ten tasks correspond to ten distinct color-coded clusters. We can observe clear boundaries between clusters of different colors, indicating that MetaAR learns distinct routing strategies for different tasks, thereby mitigating interference between them. At the same time, there are cases where clusters of different colors overlap and intersect. For example, “open window”, “open drawer”, and “open door” exhibit overlapping regions. We attribute this to MetaAR’s effective capture of the shared “open” skill among these three tasks.

Routing Path Visualization. As shown in Fig. 7, we visualize the complete routing paths over the same timestep for four tasks: “open window”, “close window”, “open drawer”, and “close drawer”. As described in Section 5.1.4, our model consists of four module layers, each containing four modules. We adopt a Top-2 routing strategy, meaning that each target module is connected only to the two most important modules from the preceding layer. In Fig. 7, dashed boxes of the same color highlight parts of the routing structure that are shared across different tasks. We observe that MetaAR effectively captures the shared object “drawer” between “open drawer” and “close drawer”, as well as the shared skill “open” between “open window” and “open drawer”. This enables knowledge sharing across tasks via shared routing structures. At the same time, each task maintains its own task-specific routing compo-

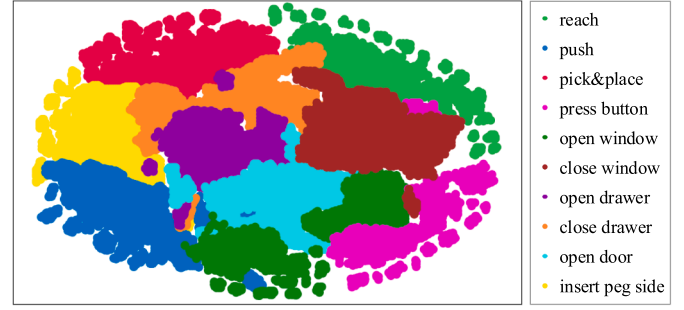


Fig. 6. t-SNE visualization for routing probability samples of different tasks under the MT10-Conditioned setting.

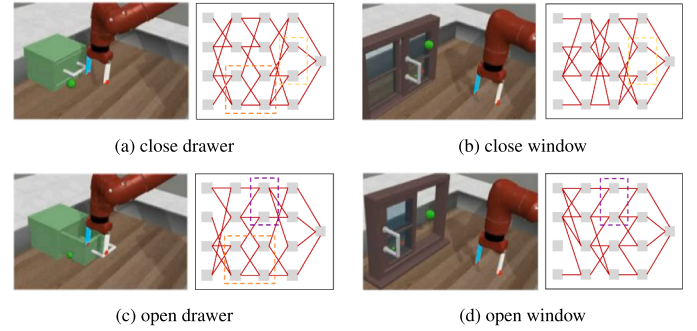


Fig. 7. Routing path visualization. A dashed box of the same color indicates modules that share the same routing across different tasks.

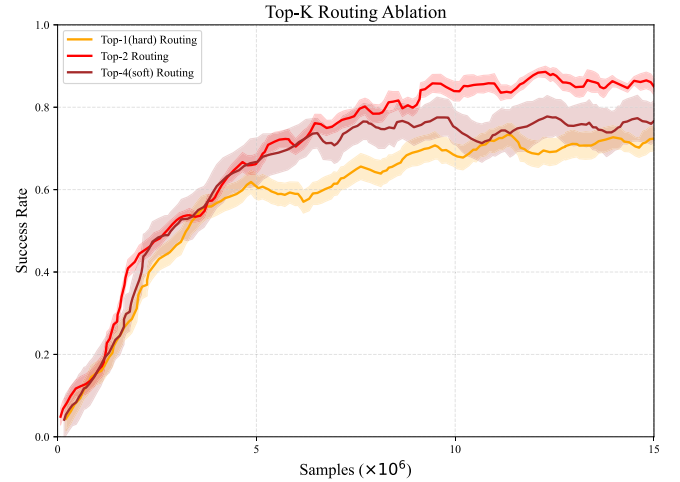


Fig. 8. Ablation experiment result of Top-K routing.

nents, demonstrating that MetaAR can effectively mitigate interference during multitask learning.

5.4. Ablation study

In this section, we construct various ablation experiments under the MT10-Conditioned setting to further analyze the effects of introducing Top-K routing and Gumbel noise, as shown in Figs. 8–10. Additionally, we also focus on the impact of the natural language metadata during the MetaAR training process, and the experiment results are shown in Fig. 11.

Effect of Top-K Routing. In the MetaAR method, to mitigate interference and select the most relevant routing path for each task, we adopt a Top-K routing strategy. To validate the effectiveness of Top-K

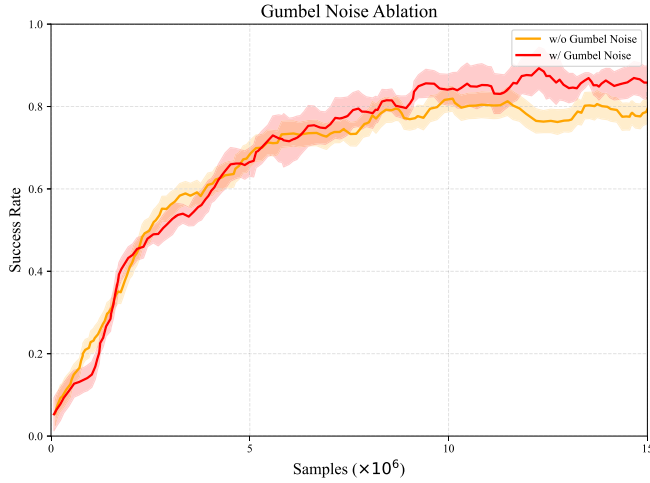


Fig. 9. Ablation experiment result of Gumbel noise. “w/ Gumbel Noise” means “with Gumbel noise” and “w/o Gumbel Noise” means “without Gumbel noise”.

routing, we conduct an ablation experiment on different values of K , as shown in Fig. 8. We further set K to 1 and 4, which are two boundary cases. Top-1 routing, also referred to as hard routing, connects each target module to only the single most relevant module from the previous layer. Top-4 routing, also known as soft routing, connects every module in one layer to all modules in the previous layer, which is the same routing mechanism used in SM [13]. As shown, the default Top-2 routing in the MetaAR method achieves the best performance. In hard routing, the single connection between modules limits knowledge fusion, resulting in ineffective knowledge sharing across tasks and reduced model performance. In contrast, soft routing establishes connections between all modules, which introduces unnecessary knowledge sharing and, consequently, interference among different tasks. These results highlight the importance of enhancing task-specific module selection.

Effect of Gumbel Noise. To investigate the role of injecting Gumbel noise in routing, we conduct an ablation study comparing two versions of MetaAR:

- MetaAR (w/ Gumbel Noise): The full model described in Section 4, where Gumbel noise is injected into the routing logits during training.
- MetaAR (w/o Gumbel Noise): A deterministic variant that removes Gumbel noise from the routing network.

As shown in Fig. 9, MetaAR without Gumbel noise exhibits smoother and faster convergence during the early training stages. However, it tends to plateau earlier, potentially missing better configurations. In contrast, MetaAR with Gumbel noise introduces greater stochasticity, resulting in higher variance in the initial phases. However, this noise injection enhances exploration by encouraging the routing network to discover and utilize more optimal module paths throughout training, ultimately leading to improved overall task performance.

Beyond the comparison in Fig. 9, we further study the effect of the Gumbel noise scale by fixing β to different values. Specifically, we compare two constant-noise settings, $\beta = 0.1$ and $\beta = 1.0$, against our default schedule where β is initialized at 1.0 and gradually decayed during training. As shown in Fig. 10, when we fix the Gumbel noise scale to a small value ($\beta = 0.1$), the training curves become smoother and more stable, but the exploration of the routing pattern space is often insufficient. Consequently, the final performance is close to that of the “no-noise” setting. In contrast, fixing a large noise scale ($\beta = 1.0$) yields better performance in the early phase of training, as the strong stochasticity promotes aggressive exploration. However, maintaining such a high level of exploration throughout training makes the learning dynamics notice-

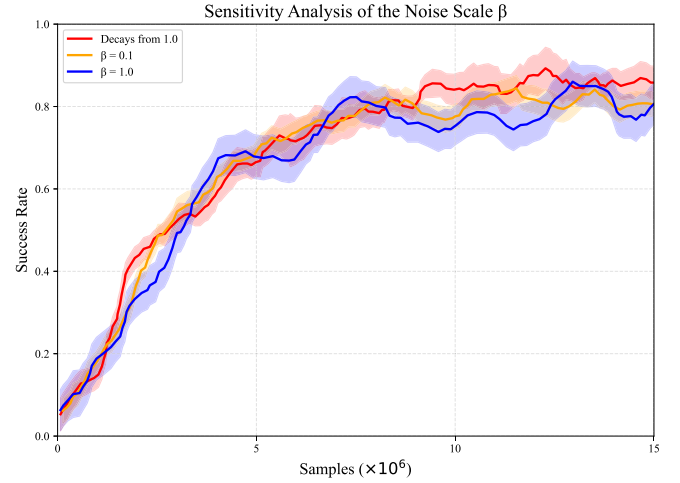


Fig. 10. The experiment result for sensitivity analysis of the noise scale β .

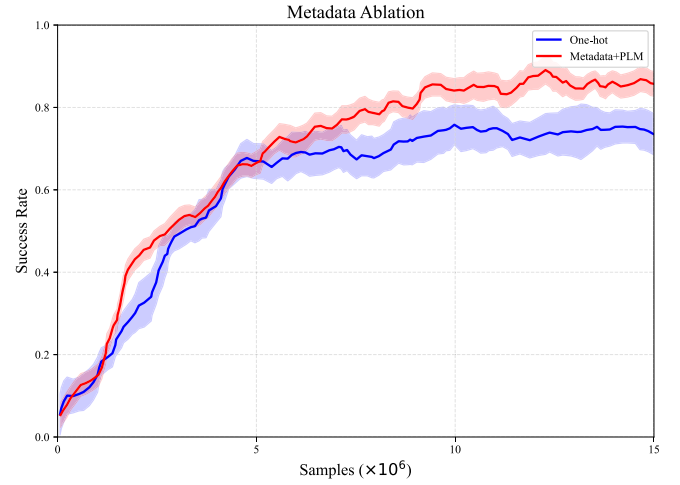


Fig. 11. Ablation experiment result of metadata.

ably noisier, the curves fluctuate more in the later phase, and the asymptotic performance becomes less stable. Overall, both constant-noise settings ($\beta = 0.1$ and $\beta = 1.0$) underperform our default schedule, where β is initialized at 1.0 and gradually decayed, confirming that the decayed scheme strikes a better balance between early exploration and late-stage stability.

Effect of Metadata. To assess the importance of metadata, we compare MetaAR within two versions that differ in how the task representation is encoded: (1) the default version that leverages natural language metadata processed via a pretrained language model (RoBERTa), and (2) a simplified version that uses a one-hot task ID encoding. Experimental results reveal notable differences, as shown in Fig. 11. The one-hot variant converges faster in training stages due to its static and discrete representation, which allows the routing network to easily memorize task-specific paths. However, this simplicity leads to limited expressiveness and poor generalization across tasks. In contrast, the metadata-based version of MetaAR leverages semantic task descriptions to facilitate the discovery of reusable and meaningful shared knowledge across related tasks. For example, in the task descriptions “Push and open a window” and “Push and close a window”, the shared skill “push” and the object “window” can be captured through the task representation, guiding the routing network toward more informed decision-making. Notably, the metadata-based MetaAR demonstrates more rapid improve-

ment in performance during early training and achieves a higher final Mean SR. These findings underscore the importance of metadata-guided task representations in enabling task-aware routing and mitigating negative knowledge interference in multitask reinforcement learning.

6. Conclusion

In this paper, we propose **Metadata-guided Adaptive Routing (MetaAR)**, a novel framework for multitask reinforcement learning that addresses the challenge of task interference and negative knowledge transfer in shared architectures. Unlike conventional approaches that rely on simple task identifiers or auxiliary embeddings, MetaAR leverages natural language task descriptions as metadata to produce expressive and interpretable task representations. These representations are fused with state information and injected into a routing network, which dynamically reconfigures the computation paths of a shared modular backbone.

To further strengthen task-specific adaptation, we introduce a noise-injected Top-K routing mechanism: for each layer, only the most relevant modules are activated while stochastic noise encourages exploration of diverse routing configurations. This design enables MetaAR to capture meaningful inter-task relationships (e.g., shared skills and objects), promote knowledge reuse across related tasks, and simultaneously reduce interference among unrelated ones, thereby striking a balance between generalization and specialization. Additionally, we evaluate MetaAR on the challenging Meta-World benchmark, covering up to 50 robotic manipulation tasks, and compare it with several strong baselines. Experimental results demonstrate that MetaAR consistently outperforms all baselines across both fixed-goal and goal-conditioned settings, achieving superior generalization, robustness, and sample efficiency.

Limitations and Future Work. In spite of these encouraging results, our study still has several limitations. First, our experiments are limited to the Meta-World benchmark with single-arm robotic manipulation and continuous-control multitask RL, so it is still unclear how MetaAR behaves in other domains such as discrete-control environments, real robotic systems, or large-scale industrial applications. Second, we strictly use the official Meta-World task descriptions as metadata and do not vary their length, specificity, or modality, which means we do not yet provide a systematic analysis of how different forms of metadata affect performance. Third, we focus on single-agent settings and do not consider partial observability or multi-agent interactions, where metadata might play different roles in coordination and credit assignment. These aspects point to several promising directions for future research. One direction is to perform a systematic study of metadata design by varying the quality, granularity, and modality of task descriptions, and to quantify how these choices influence task separation, routing patterns, and overall performance in MetaAR. Another direction is to explore the impact of different routing granularities, such as module-level, layer-level, or neuron-level routing, on the trade-off between knowledge sharing and task isolation in multitask reinforcement learning. Finally, it would be interesting to extend MetaAR to broader domains, including tasks that jointly involve vision, language, and action or multi-agent reinforcement learning scenarios, where richer metadata such as visual context or symbolic instructions could further enhance metadata-guided adaptive architectures.

CRedit authorship contribution statement

Rui Pan: Writing – original draft, Validation, Software, Methodology, Data curation, Conceptualization; **Haoran Luo:** Visualization, Software; **Quan Yuan:** Writing – review & editing, Supervision; **Guiyang Luo:** Writing – review & editing, Supervision; **Jinglin Li:** Supervision, Funding acquisition; **Tiesunlong Shen:** Writing – review & editing, Supervision; **Rui Mao:** Writing – review & editing, Supervision; **Erik Cambria:** Writing – review & editing, Supervision.

Data availability

The data used in this study are publicly available.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work is supported in part by the [Natural Science Foundation of China](#) under Grant 62272053 and Grant 62472048, in part by the [Beijing Nova Program](#) under Grant 20230484364, and in part by [Beijing Natural Science Foundation](#) under Grant L242081.

References

- [1] R. Zhang, C. Zhao, H. Du, D. Niyato, J. Wang, S. Sawadsitang, X. Shen, D.I. Kim, Embodied ai-enhanced vehicular networks: an integrated vision language models and reinforcement learning method, *IEEE Trans. Mob. Comput.* (2025).
- [2] J. Guo, X. Li, Z. Liu, J. Ma, C. Yang, J. Zhang, D. Wu, TROVE: a context-awareness trust model for VANETs using reinforcement learning, *IEEE Internet Things J.* 7 (7) (2020) 6647–6662.
- [3] B.S. Sarikaya, Ş. Bahtiyar, A survey on security of UAV and deep reinforcement learning, *Ad Hoc Netw.* 164 (2024) 103642.
- [4] J. Guo, H. Gao, Z. Liu, F. Huang, J. Zhang, X. Li, J. Ma, ICRA: An intelligent clustering routing approach for UAV ad hoc networks, *IEEE Trans. Intell. Transp. Syst.* 24 (2) (2022) 2447–2460.
- [5] S. Latif, H. Cuayáhuil, F. Pervez, F. Shamsad, H.S. Ali, E. Cambria, A survey on deep reinforcement learning for audio-based applications, *Artif. Intell. Rev.* 56 (2023) 2193–2240.
- [6] H. Peng, Y. Ma, S. Poria, Y. Li, E. Cambria, Phonetic-enriched text representation for chinese sentiment analysis with reinforcement learning, *Inf. Fusion* 70 (2021) 88–99.
- [7] K. Zhang, Y. Li, J. Wang, E. Cambria, X. Li, Real-time video emotion recognition based on reinforcement learning and domain knowledge, *IEEE Trans. Circuits Syst. Video Technol.* 32 (3) (2021) 1034–1047.
- [8] N. Vithayathil Varghese, Q.H. Mahmoud, A survey of multi-task deep reinforcement learning, *Electronics* 9 (9) (2020) 1363.
- [9] Y. Li, Q. Pan, E. Cambria, Deep-attack over the deep reinforcement learning, *Knowl. Based Syst.* 250 (2022) 108965.
- [10] Y. Teh, V. Bapst, W.M. Czarnecki, J. Quan, J. Kirkpatrick, R. Hadsell, N. Heess, R. Pascanu, Distral: robust multitask reinforcement learning, *Adv. Neural Inf. Process. Syst.* 30 (2017).
- [11] M. Hessel, H. Soyer, L. Espeholt, W. Czarnecki, S. Schmitt, H. Van Hasselt, Multi-task deep reinforcement learning with popart, in: *Proceedings of AAAI*, 33, 2019, pp. 3796–3803.
- [12] M. Cho, W. Jung, Y. Sung, Multi-task reinforcement learning with task representation method, in: *ICLR 2022 Workshop on Generalizable Policy Learning in Physical World*, 2022.
- [13] R. Yang, H. Xu, Y. Wu, X. Wang, Multi-task reinforcement learning with soft modularization, *Adv. Neural Inf. Process. Syst.* 33 (2020) 4767–4777.
- [14] E. Parisotto, J.L. Ba, R. Salakhutdinov, Actor-mimic: deep multitask and transfer reinforcement learning, *arXiv preprint arXiv:1511.06342* (2015).
- [15] G. Shi, Q. Li, W. Zhang, J. Chen, X.M. Wu, Recon: reducing conflicting gradients from the root for multi-task learning, *arXiv preprint arXiv:2302.11289* (2023).
- [16] T. Yu, S. Kumar, A. Gupta, S. Levine, K. Hausman, C. Finn, Gradient surgery for multi-task learning, *Adv. Neural Inf. Process. Syst.* 33 (2020) 5824–5836.
- [17] J. He, K. Li, Y. Zang, H. Fu, Q. Fu, J. Xing, J. Cheng, Not all tasks are equally difficult: multi-task deep reinforcement learning with dynamic depth routing, in: *Proceedings of the AAAI Conference on Artificial Intelligence*, 38, 2024, pp. 12376–12384.
- [18] S. Sodhani, A. Zhang, J. Pineau, Multi-task reinforcement learning with context-based representations, in: *International Conference on Machine Learning, PMLR*, 2021, pp. 9767–9779.
- [19] T. Yu, D. Quillen, Z. He, R. Julian, K. Hausman, C. Finn, S. Levine, Meta-world: a benchmark and evaluation for multi-task and meta reinforcement learning, in: *Conference on Robot Learning, PMLR*, 2020, pp. 1094–1100.
- [20] N. Shazeer, A. Mirhoseini, K. Maziarz, A. Davis, Q. Le, G. Hinton, J. Dean, Outrageously large neural networks: the sparsely-gated mixture-of-experts layer, *arXiv preprint:1701.06538* (2017).
- [21] J. Feng, M. Chen, Z. Pu, T. Qiu, J. Yi, J. Zhang, Efficient multi-task reinforcement learning via task-specific action correction, *IEEE Trans. Cognit. Dev. Syst.* (2025).
- [22] M.R. Samsami, H. Alimadad, Distributed deep reinforcement learning: an overview, *arXiv preprint:2011.11012* (2020).
- [23] A. Hendawy, J. Peters, C. D'Eramo, Multi-task reinforcement learning with mixture of orthogonal experts, *arXiv preprint:2311.11385* (2023).
- [24] L. Sun, H. Zhang, W. Xu, M. Tomizuka, Paco: parameter-compositional multi-task reinforcement learning, *Adv. Neural Inf. Process. Syst.* 35 (2022) 21495–21507.

- [25] G. Cheng, L. Dong, W. Cai, C. Sun, Multi-task reinforcement learning with attention-based mixture of experts, *IEEE Rob. Autom. Lett.* 8 (6) (2023) 3812–3819.
- [26] X. Sun, R. Panda, R. Feris, K. Saenko, Adashare: learning what to share for efficient deep multi-task learning, *Adv. Neural Inf. Process. Syst.* 33 (2020) 8728–8740.
- [27] T. Shen, R. Mao, J. Wang, X. Zhang, E. Cambria, Flow-guided direct preference optimization for knowledge graph reasoning with trees, in: *Proceedings of the 48th International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2025, pp. 1165–1175.
- [28] M. Firdaus, A. Ekbal, E. Cambria, Multitask learning for multilingual intent detection and slot filling in dialogue systems, *Inf. Fusion* 91 (2023) 299–315.
- [29] B.R. Wu, S. Ormazabal Arriagada, T.C. Hsu, T.W. Lin, C. Lin, Exploiting common patterns in diverse cancer types via multi-task learning, *npj Precis. Oncol.* 8 (1) (2024) 245.
- [30] A. Jaspal, F. Zhang, W. Chang, S. Kumar, Y. Wang, R. Mittleman, Q. Wang, W. Mao, Balancing semantic relevance and engagement in related video recommendations, *arXiv preprint:2507.09403* (2025).
- [31] G. Lyu, W. Kang, H. Wang, Z. Li, Z. Yang, S. Feng, Common-individual semantic fusion for multi-view multi-label learning, in: *International Joint Conference on Artificial Intelligence*, 2024, pp. 4715–4723.
- [32] R. Upadhyay, R. Phlypo, R. Saini, M. Liwicki, Meta-sparsity: learning optimal sparse structures in multi-task networks through meta-learning, *arXiv preprint:2501.12115* (2025).
- [33] X. Zhang, R. Mao, E. Cambria, Granular syntax processing with multi-task and curriculum learning, *Cognit. Comput.* 16 (6) (2024) 3020–3034.
- [34] Y. Ma, R. Mao, Q. Lin, P. Wu, E. Cambria, Quantitative stock portfolio optimization by multi-task learning risk and return, *Inf. Fusion* 104 (2024) 102165.
- [35] V. Bellon, V. Stoven, C.A. Azencott, Multitask feature selection with task descriptors, in: *Biocomputing 2016: Proceedings of the Pacific Symposium*, World Scientific, 2016, pp. 261–272.
- [36] Z. Zheng, Y. Wang, Q. Dai, H. Zheng, D. Wang, Metadata-driven task relation discovery for multi-task learning, in: *IJCAI*, 2019, pp. 4426–4432.
- [37] J. Yan, F. He, Q. Sang, B. Tong, P. Sun, Y. Gong, C. Hu, D. Cheng, Metadata-guided adaptable frequency scaling across heterogeneous applications and devices, *arXiv preprint:2509.22707* (2025).
- [38] A. Hallak, D. Di Castro, S. Mannor, Contextual markov decision processes, *arXiv preprint:1502.02259* (2015).
- [39] H. Lane, M. Dyshel, *Natural Language Processing in Action*, Simon and Schuster, 2025.
- [40] S.P. Singh, Transfer of learning by composing solutions of elemental sequential tasks, *Mach. Learn.* 8 (1992) 323–339.
- [41] Q.W. Khan, Exploring Markov decision processes: a comprehensive survey of optimization applications and techniques, *Igmin Res.* 2 (7) (2024) 508–517.
- [42] T. Haarnoja, A. Zhou, P. Abbeel, S. Levine, Soft actor-critic: off-policy maximum entropy deep reinforcement learning with a stochastic actor, in: *International Conference on Machine Learning*, Pmlr, 2018, pp. 1861–1870.
- [43] S. Huang, Z. Ma, J. Du, C. Meng, W. Wang, J. Leng, M. Guo, Z. Lin, Gumbel reranking: differentiable end-to-end reranker optimization, *arXiv preprint:2502.11116* (2025).
- [44] Y. Liu, M. Ott, N. Goyal, J. Du, M. Joshi, D. Chen, O. Levy, M. Lewis, L. Zettlemoyer, V. Stoyanov, Roberta: a robustly optimized bert pretraining approach, *arXiv preprint:1907.11692* (2019).
- [45] H.J. Jeon, *Information-Theoretic Foundations for Machine Learning*, Ph.D. thesis, Stanford University, 2025.
- [46] N. Kumar, A. Dixit, V. Vijay, Entropy measures and their applications: a comprehensive review, *arXiv preprint:2503.15660* (2025).
- [47] K. Zakka, B. Tabanpour, Q. Liao, M. Haiderbhai, S. Holt, J.Y. Luo, A. Allshire, E. Frey, K. Sreenath, L.A. Kahrs, et al., Mujoco playground, *arXiv preprint:2502.08844* (2025).
- [48] S. Jung, T. Dagobert, J.M. Morel, G. Facciolo, A review of t-SNE, *Image Processing On Line* 14 (2024) 250–270.